Занятие 4: Валидация, бизнес-правила и обработка ошибок

Проверка данных, бизнес-логика, контракты ошибок

Слайд 1: Зачем нужна валидация

Защита от некорректных данных

- Предотвращение ошибок в приложении
- Сохранение целостности данных

Безопасность

- SQL injection
- XSS
- CSRF

Улучшение UX

- Понятные сообщения об ошибках
- Быстрое обнаружение проблем

Соответствие контракту

- API должен работать предсказуемо
- Клиент ожидает определённый формат

Слайд 2: Типы валидации

Тип данных

- Проверка типа (int, str, bool)
- Пример: возраст должен быть числом

Формат

- Email, URL, телефон
- Регулярные выражения
- Пример: email должен содержать @

Обязательность

- Обязательные поля
- Пример: имя обязательно

Диапазон значений

- Минимум/максимум
- Пример: возраст 18-120

Слайд 3: Отсутствующее поле vs null vs пустая строка vs ноль

Отсутствующее поле

```
{
  "name": "John"
  // email отсутствует
}
```

Null

```
{
  "name": "John",
  "email": null
}
```

Пустая строка

```
{
  "name": "John",
  "email": ""
}
```

Ноль

```
{
  "name": "John",
  "age": 0
}
```

Слайд 4: Проверка формата vs бизнес-правила

Проверка формата

- Синтаксическая проверка
- Не зависит от контекста
- Пример: email содержит @ и .

Бизнес-правила

- Семантическая проверка
- Зависит от контекста
- Пример: email уникален, возраст >= 18

Пример:

```
# Формат
if '@' not in email:
    raise ValidationError("Invalid email format")

# Бизнес-правило
if User.exists(email=email):
    raise ValidationError("Email already exists")
```

Слайд 5: Почему проверки нужны на сервере

Клиентскую валидацию можно обойти

- curl, Postman, прямые запросы
- Отключённый JavaScript
- Модифицированный клиент

Безопасность

- Защита от атак
- Сохранение целостности данных

Надёжность

- Единый источник правды
- Гарантия корректности данных

Пример:

```
// Клиент (можно обойти)
if (age < 18) {
  showError("Too young");
}

// Сервер (нельзя обойти)
if age < 18:
  raise ValidationError("Age must be 18+")
```

Слайд 6: Типы ошибок

Ошибки клиента (4xx)

- Некорректный запрос (400)
- Ошибка валидации (422)
- Не найден (404)
- Конфликт (409)

Конфликты бизнес-правил

- Дубликат ресурса (409)
- Нарушение ограничений
- Недостаточно прав (403)

Внутренние сбои (5xx)

- Ошибка базы данных (500)
- Ошибка внешнего сервиса (502)
- Тайм-аут (504)

Слайд 7: Понятный контракт ошибки

Структура ошибки:

```
{
  "error": "Validation error",
  "message": "Email is invalid",
  "details": {
    "field": "email",
    "reason": "Invalid format"
  },
  "code": "VALIDATION_ERROR"
}
```

Что сообщать клиенту:

- Что пошло не так
- Как исправить
- Код ошибки для программной обработки

Что оставлять в логах:

- Полный стек вызовов
- Внутренние детали
- Контекст запроса

Слайд 8: Обычные, граничные и ошибочные сценарии

Обычные сценарии (Happy Path)

- Корректные данные
- Всё работает как ожидается
- Пример: создание пользователя с валидными данными

Граничные сценарии (Edge Cases)

- Минимальные/максимальные значения
- Пустые списки
- Граничные условия
- Пример: возраст = 18, возраст = 120

Ошибочные сценарии (Error Cases)

- Некорректные данные
- Отсутствующие ресурсы
- Нарушение правил
- Пример: email = "", возраст = 200

Слайд 9: Пример валидации

Запрос:

```
{
  "name": "J",
  "email": "invalid-email",
  "age": 15
}
```

Ошибки:

```
{
  "error": "Validation error",
  "details": {
    "name": "Name must be at least 2 characters",
    "email": "Invalid email format",
    "age": "Age must be at least 18"
  }
}
```

Статус: 400 Bad Request

Слайд 10: Пример бизнес-правила

Запрос:

```
{
  "email": "existing@example.com"
}
```

Ошибка:

```
{
  "error": "Conflict",
  "message": "Email already exists",
  "details": {
    "field": "email",
    "value": "existing@example.com"
  }
}
```

Статус: 409 Conflict

Слайд 11: Обработка ошибок на сервере

Псевдокод:

```
def create_user(request):
    try:
        # Валидация формата
        validate_format(request.data)

        # Валидация бизнес-правил
        validate_business_rules(request.data)

        # Создание пользователя
        user = User.create(request.data)

        return user, 201

    except ValidationError as e:
        return {"error": str(e)}, 400

    except BusinessRuleError as e:
        return {"error": str(e)}, 409

    except Exception as e:
        log_error(e)
        return {"error": "Internal error"}, 500
```

Слайд 12: Логирование ошибок

Что логировать:

- Стек вызовов
- Контекст запроса
- Параметры (без секретов)
- Время выполнения

Что НЕ логировать:

- Пароли
- Токены авторизации
- Персональные данные (если не нужно)

Уровни логирования:

- ERROR - критические ошибки
- WARN - потенциальные проблемы
- INFO - важные события
- DEBUG - отладочная информация

Слайд 13: Практическое задание

Задание 1: Валидация
Дан запрос с некорректными данными. Определите:

- Какие проверки не пройдут
- Какие статусы вернуть
- Какие сообщения об ошибках

Задание 2: Бизнес-правила
Определите бизнес-правила для:

- Регистрации пользователя
- Создания заказа
- Добавления комментария

Задание 3: Контракт ошибки
Создайте структуру ошибки для:

- Валидации
- Конфликта
- Внутренней ошибки

Слайд 14: Вопрос для собеседования

Вопрос: Разберите некорректную заявку и определите, на каком этапе и почему её отклонить.

Дано:

```
{
  "email": "test@test",
  "age": 17,
  "password": "123"
}
```

Ожидаемый ответ:

1. Проверка формата email: неверный формат
2. Проверка возраста: меньше 18
3. Проверка пароля: слишком короткий
4. Отклонить на этапе валидации
5. Статус 400 с деталями ошибок